

지지흐름 텐서장(SFTF) 쉽게 읽기

3D 프린팅에서 “어느 방향으로 출력할까”를 빠르게 고르는 방법

—— 고등학생을 위한 수식 한 줄씩 풀어 읽기 ——

이 글의 목표. 원래 논문(*Support Flow Tensor Field*, 줄여서 **SFTF**)에 나오는 수식들이 대학원생도 한 번에 이해하기 어렵다는 이야기를 들었습니다. 그래서 이 글에서는 같은 내용을, 고등학교 1~2학년이 아는 수학(벡터, 내적, 평균, 그래프)만으로 읽을 수 있도록 수식 한 줄마다 바로 밑에 우리말 설명을 붙였습니다. 처음 보는 용어는 각주^a로 최대한 많이 풀었습니다. 수학이 부담되면 파란 박스(“한 줄 요약”)만 따라 읽어도 전체 흐름을 잡을 수 있습니다.

^a각주(脚註)란 이렇게 페이지 아래쪽에 작은 글씨로 다는 보충 설명입니다. 본문 흐름을 끊지 않으면서 어려운 단어를 따로 풀어 주는 장치입니다.

Contents

1	먼저 큰 그림: 무슨 문제를 푸는가	2
1.1	3D 프린팅과 “서포트”	2
1.2	왜 “방향”이 중요한가	2
1.3	SFTF의 아이디어 한 문장	2
2	준비물: 고등학교 수학으로 충분합니다	2
2.1	면, 면적, 법선벡터	3
2.2	내적: 두 방향이 “얼마나 같은 쪽”인지 재는 자	3
2.3	평균을 모으면 “전체 경향”이 보인다	3
3	출발점: 대칭 지지 텐서 (그리고 PCA 이야기)	3
4	SFTF의 핵심 수식 (한 줄씩)	4
4.1	1단계 — 이 면이 “치진 면”인가? (오버행 계수)	4
4.2	2단계 — 누가 이 면을 받쳐 주나? (레이캐스트)	5
4.3	3단계 — 모든 받침 관계를 하나로 모으기 (흐름 텐서)	5
4.4	보너스 — 계산을 너무 무겁지 않게 (레이 예산)	6
5	가장 중요한 항: 바닥판(Ground Node)	6
5.1	바닥판 받침의 양	6
5.2	멋진 통합: 바닥판도 “가상의 받침면”으로 보기	7
5.3	레일리 값: 텐서를 하나의 비용 숫자로	7
6	최종 점수와 후보 고르기	7
6.1	기본 점수 J	7
6.2	후보를 만드는 절차	8
6.3	순위 정규화로 최종 정렬 (단위가 다른 값들을 공정하게)	8
7	후보를 “검증”하고, 어려우면 “더 본다” (AVE)	8
7.1	국소 정밀검증	8
7.2	어려운 경우 자동 확대 (AVE)	9

8	결과 요약과 정직한 한계	9
8.1	얼마나 빠르냐	9
8.2	어떤 향이 가장 쓸모 있냐	9
8.3	정직한 한계 (이게 제일 중요)	10

1 먼저 큰 그림: 무슨 문제를 푸는가

1.1 3D 프린팅과 “서포트”

FDM¹ 같은 3D 프린터는 재료를 아래에서 위로 한 층씩 쌓아 물체를 만듭니다. 그런데 공중에 떠 있거나 심하게 기울어진 부분(오버행²)은 그냥 두면 흘러내립니다. 그래서 프린터는 임시 기둥인 서포트(support, 받침)를 함께 출력해 받쳐 주고, 출력이 끝나면 이 서포트를 뜯어 버립니다.

핵심 갈등. 서포트는 버리는 재료입니다. 재료비 · 출력시간 · 뒤처리 수고를 모두 늘립니다. 그러니 서포트가 적게 생기도록 출력하고 싶습니다.

1.2 왜 “방향”이 중요한가

같은 물체라도 어떤 자세(방향)로 눕혀서 출력하느냐에 따라 필요한 서포트의 양이 크게 달라집니다. 킥을 똑바로 세워 출력하면 손잡이 아래가 떠서 서포트가 많이 필요하지만, 옆으로 눕히면 줄어든 수도 있습니다. 그래서 “서포트가 가장 적게 드는 출력 방향”을 찾는 것이 중요한 문제입니다.

이 “방향”을 수학에서는 단위벡터³ n 으로 적습니다. 3차원 공간에서 길이가 1인 화살표는 공의 표면(반지름 1짜리 구) 위의 한 점에 대응하므로, 가능한 모든 방향의 모임을 S^2 ⁴라고 씁니다. 즉 $n \in S^2$ 는 “ n 은 어떤 한 방향”이라는 뜻입니다.

무식한 방법(전수조사)의 문제. “그럼 모든 방향을 1°씩 다 돌려보며 서포트 양을 재면 되지 않나?” 맞지만, 방향의 가짓수가 수만 개라서 모든 방향마다 정밀 계산을 돌리면 매우 느립니다. 단순한 모양조차 시간이 오래 걸립니다.

1.3 SFTF의 아이디어 한 문장

한 줄 요약. SFTF는 정답을 직접 푸는 정밀 계산기가 아니라, “이 방향들이 유망해 보인다”를 아주 빠르고 값싸게 골라 주는 후보 추천기입니다. 그다음 추천된 몇 개 방향 주변만 정밀 검증하면, 전부 다 검사할 때와 비슷한 좋은 답을, 훨씬 적은 계산으로 얻습니다.

비유하면, 넓은 바다에서 보물을 찾을 때 바다 전체를 잠수해 뒤지는 대신(전수조사), 먼저 수면 위에서 “여기 색이 좀 다른데?” 하는 곳 20군데를 빠르게 찍고(SFTF), 그 20군데 주변만 잠수해 확인하는(국소 정밀 검증) 전략입니다.

2 준비물: 고등학교 수학으로 충분합니다

본격적인 수식에 들어가기 전에, 앞으로 계속 쓰는 도구 세 가지만 짚겠습니다.

¹FDM(Fused Deposition Modeling, 용융 적층 모델링): 가는 플라스틱 실을 녹여 한 층(layer)씩 쌓아 올려 입체를 만드는 가장 흔한 3D 프린팅 방식입니다. 케이크에 생크림을 짜서 한 겹씩 올리는 모습을 떠올리면 됩니다.

²오버행(overhang, 처마): 아래에 받쳐 주는 것이 없는데 옆이나 위로 튀어나온 부분. 처마나 발코니처럼요. 너무 기울면 갓 쌓은 재료가 굳기 전에 흘러내려 모양이 망가집니다.

³벡터(vector): 크기와 방향을 함께 가진 양. 화살표로 그립니다. 단위벡터는 길이(크기)가 정확히 1인 벡터로, “방향만” 나타낼 때 씁니다.

⁴ S^2 : “2차원 구면(sphere)”이라는 기호. 3차원 공간 안에 있는, 반지름 1인 공의 껍질을 뜻합니다. 지구본 표면 위의 점 하나가 “한 방향”이라고 생각하면 됩니다.

2.1 면, 면적, 법선벡터

3D 모델은 보통 수많은 삼각형 조각으로 이루어진 껍질(메쉬⁵)입니다. 각 삼각형 면 i 마다 두 가지 숫자를 씁니다.

- A_i : 그 삼각형의 면적(넓이). 큰 면일수록 더 중요하게 다루려고 씁니다.
- m_i : 그 삼각형의 법선벡터⁶. 길이 1짜리 화살표로, 그 면이 “어느 쪽을 바라보는지”를 알려 줍니다.

2.2 내적: 두 방향이 “얼마나 같은 쪽”인지 재는 자

두 단위벡터 a, b 의 내적⁷ $a \cdot b$ 은 다음과 같이 읽으면 됩니다.

$$a \cdot b = \cos \theta \quad (\theta = \text{두 화살표 사이의 각도}) \quad (1)$$

읽기: 두 방향이 완전히 같으면 $a \cdot b = 1$, 직각이면 0, 정반대면 -1 입니다. 즉 내적은 “두 화살표가 얼마나 같은 쪽을 가리키나”를 하나의 숫자로 압축한 것입니다. 이 한 줄만 기억하면 뒤의 수식 대부분이 풀립니다.

2.3 평균을 모으면 “전체 경향”이 보인다

수많은 면의 정보를 하나하나 보는 대신, 전부 더해서(합) 전체 경향을 보는 일을 자주 합니다. 기호 $\sum$ 은 “모두 더하라”는 뜻입니다⁸. 예를 들어 $\sum_i A_i$ 는 “모든 삼각형 면적을 다 더하라” = 표면 전체 넓이입니다.

3 출발점: 대칭 지지 텐서 (그리고 PCA 이야기)

SFTF를 만들기 전에, 먼저 더 단순한 “기준선(baseline)”⁹ 하나를 봅시다. 면의 법선들이 전체적으로 어느 쪽으로 쏠려 있는지를 요약하는 행렬입니다.

$$M = \sum_i A_i m_i m_i^T \quad (2)$$

한 줄씩 읽기.

- $\sum_i$: 모든 삼각형 면에 대해 더한다.
- A_i : 그 면의 넓이로 가중치¹⁰를 준다. 큰 면일수록 더 크게 반영.
- $m_i m_i^T$: 법선벡터 자기 자신을 “표 형태”로 곱한 것(외적¹¹). 이 작은 표들을 모두 더하면, 표면 법선이 주로 어느 방향으로 퍼져 있는지를 요약하는 하나의 큰 표 M 이 됩니다.

⁵메쉬(mesh, 그물): 3D 물체의 표면을 잘게 쪼갠 삼각형들의 모임. 삼각형이 많을수록(=면 개수가 많을수록) 표면이 매끄럽고 정밀하지만 계산은 무거워집니다.

⁶법선벡터(normal vector): 면에 수직으로 바깥을 향해 튀어나온 단위벡터. 책상 위에 종이를 놓으면 천장을 향하는 화살표가 그 종이의 법선입니다. “이 면이 어느 쪽을 바라보는가”를 나타냅니다.

⁷내적(dot product, 점곱): 두 벡터를 곱해 하나의 숫자를 얻는 연산. 성분이 $a = (a_1, a_2, a_3), b = (b_1, b_2, b_3)$ 이면 $a \cdot b = a_1 b_1 + a_2 b_2 + a_3 b_3$. 단위벡터끼리는 $a \cdot b = \cos \theta$ (θ =두 벡터가 이루는 각)가 되어, 방향이 얼마나 비슷한지를 $-1 \sim 1$ 사이 숫자로 알려 줍니다.

⁸ $\sum_i x_i$ (i 에 대한 합, 시그마): $x_1 + x_2 + x_3 + \dots$ 처럼 여러 항을 전부 더하라는 기호. $\sum_{i=1}^3 x_i = x_1 + x_2 + x_3$.

⁹기준선(baseline): 성능을 비교하기 위한 출발점 또는 비교 대상. “적어도 이것보단 나아야 한다”는 잣대입니다.

¹⁰가중치(weight): “얼마나 비중 있게 칠지”를 정하는 곱하는 수. 큰 면일수록 전체 경향에 더 크게 반영되도록 면적 A_i 를 곱해 줍니다. 학교 성적에서 중간고사보다 기말고사 비중을 크게 두는 것과 같은 원리입니다.

¹¹ mm^T (외적, outer product): 세로 벡터 m 과 가로 벡터 m^T (T 는 전치, 즉 세로를 가로로 눕힘¹²)를 곱해 3×3 정사각형 표(행렬)를 만드는 연산. 이 표 하나가 “그 면이 가리키는 방향”의 정보를 담습니다. 내적($m \cdot m$ = 숫자 하나)과 헷갈리지 마세요. 외적은 표, 내적은 숫자입니다.

¹²전치(transpose, T): 행렬의 행과 열을 맞바꾸는 것. 세로로 쓴 숫자들을 가로로 눕히는 조작이라고 생각하면 됩니다.

이 M 이 바로 PCA와 같은 원리입니다. PCA^a는 데이터가 “가장 많이 퍼진 방향”을 찾는 기법인데, 그 계산의 핵심이 바로 식 (2)처럼 (데이터) $\times$ (데이터)^T 를 모두 더한 표 (공분산행렬^b) 를 만든 뒤, 그 표의 고유벡터^c를 구하는 것이기 때문입니다. 그래서 M 의 고유벡터는 “이 물체의 표면이 전체적으로 향하는 대표 방향”을 알려 줍니다.

^aPCA(Principal Component Analysis, 주성분 분석): 여러 방향으로 흩어진 데이터에서 가장 많이 퍼진 주된 방향을 찾아내는 통계 기법. 예를 들어 키-몸무게 점들이 비스듬한 타원으로 퍼져 있으면, PCA는 그 타원의 긴 축 방향을 “첫 번째 주성분”으로 찾아 줍니다. “데이터가 가장 길게 늘어선 방향”을 찾는 도구라고 보면 됩니다.

^b공분산행렬(covariance matrix): 여러 변수(또는 방향)가 서로 얼마나 함께 변하는지를 한 표에 모은 것. PCA 는 이 표의 고유벡터를 구해 주성분 방향을 얻습니다.

^c고유벡터 (eigenvector) · 고유값(eigenvalue): 어떤 행렬 M 을 곱해도 방향이 바뀌지 않고 길이만 변하는 특별한 벡터가 고유벡터, 그때 늘어난 배율이 고유값입니다. ($Mv = \lambda v$, 여기서 v =고유벡터, λ =고유값.) PCA 에서 가장 큰 고유값에 해당하는 고유벡터가 “데이터가 가장 많이 퍼진 주된 방향”입니다.

그런데 이걸로는 부족합니다. M 은 면들이 전체적으로 어디를 보는지만 요약할 뿐, 정작 우리가 알고 싶은 것 — (1) 어떤 면이 특정 출력 방향에서 오버행인가, (2) 그 면을 실제로 받쳐 줄 다른 면이 있는가, (3) 못 받치면 바닥판까지 얼마나 먼가 — 는 전혀 답지 못합니다. 게다가 M 은 방향을 바꿔 넣어도 답이 같은 대칭적¹³ 양이라, “위로 향한 면”과 “아래로 처진 면”을 구별하지 못합니다. 바로 이 약점을 고치려고 SFTF가 등장합니다.

4 SFTF의 핵심 수식 (한 줄씩)

이제 SFTF가 하나의 출력 방향 n 에 대해 “서포트가 얼마나 들지”를 어떻게 점수로 매기는지 차례로 봅시다. 핵심 발상은 지지(받침)를 “흐름”으로 보는 것입니다 — 마치 물이 위에서 아래로 흐르듯, “받쳐야 할 무게”가 처진 면에서 아래의 받침면(또는 바닥)으로 흘러내린다고 상상합니다.

4.1 1단계 — 이 면이 “처진 면”인가? (오버행 계수)

$$O_i(n) = \max(0, -m_i \cdot n) \quad (3)$$

한 줄씩 읽기.

- n : 우리가 시험 중인 출력 방향(위쪽 방향). m_i : 면 i 의 법선 (그 면이 바라보는 쪽).
- $m_i \cdot n$: 면이 출력 방향과 같은 쪽을 보면 양수(+), 반대쪽 (즉 아래로)을 보면 음수(-). (앞서 배운 “내적 = 방향 비슷한 정도”!)
- $-m_i \cdot n$: 부호를 뒤집었으니, 아래로 처진 면일수록 큰 양수가 됩니다.
- $\max(0, \cdot)$: 음수면 그냥 0 으로 만든다는 뜻¹⁴. 즉 위로 향한 면(서포트가 필요 없는 면)은 0점 처리하고, 아래로 처진 면만 점수를 줍니다.

한 줄 요약. $O_i(n)$ 은 “면 i 가 이 출력 방향에서 얼마나 심하게 아래로 처졌나”를 나타내는 점수입니다. 0 이면 안 처진 것, 클수록 심하게 처진 오버행입니다.

¹³대칭 (symmetric): 여기서는 “위/아래, 앞/뒤를 뒤집어도 결과가 똑같다”는 뜻. 하지만 3D 프린팅에서 위로 향한 면과 아래로 처진 면은 완전히 다릅니다 (아래로 처진 면만 서포트가 필요!). 그래서 위아래를 구별하지 못하는 대칭적인 M 으로는 부족합니다.

¹⁴ $\max(0, x)$: x 가 양수면 그대로 두고, 음수면 0으로 바꾸는 함수. “나쁜(음수) 값은 무시하고 좋은(양수) 값만 센다”는 장치. 머신러닝에서 ReLU라고도 부릅니다.

4.2 2단계 — 누가 이 면을 받쳐 주나? (레이캐스트)

처진 면 i 를 발견했으면, “그 아래에 받쳐 줄 다른 면이 있나?”를 봐야 합니다. 이를 위해 면 i 에서 아래쪽($-n$ 방향)으로 가상의 직선(광선)을 쏩니다. 이를 **레이캐스트**¹⁵라고 합니다. 빛줄기가 처음 맞은 면을 받침면 j 라고 합니다. 단, 아무 면이나 인정하지 않고 세 가지 조건을 통과해야 합니다.

- 자기 자신은 안 됨 ($j \neq i$).
- 받침면이 처진 면보다 아래에 있어야 함 (높이차 $h_{ij} > 0$).
- 받침면이 위를 보고 있어야 함 ($m_j \cdot n > 0.05$, 즉 어느 정도 하늘을 향해야 받칠 수 있음).

받침면을 찾았으면, 둘 사이의 높이차와 영향력을 계산합니다.

$$h_{ij}(n) = (c_i - c_j) \cdot n \quad (4)$$

읽기: c_i, c_j 는 두 면의 중심점¹⁶ 위치입니다. 두 점의 위치 차이를 출력 방향 n 에 내적했으니, h_{ij} 는 “처진 면이 받침면보다 출력 방향으로 얼마나 높이 떠 있나” = 둘 사이의 세로 거리입니다.

$$w_{ij}(n) = \frac{O_i(n)}{1 + \alpha h_{ij}(n)}, \quad \alpha = \frac{1}{D} \quad (5)$$

한 줄씩 읽기.

- 분자 $O_i(n)$: 면이 심하게 처질수록(=받쳐야 할 게 많을수록) 영향력 $\uparrow$.
- 분모 $1 + \alpha h_{ij}$: 받침면이 멀수록(높이차 h_{ij} 가 클수록) 분모가 커져서 영향력 $\downarrow$. 즉 멀리 떨어진 받침은 덜 쳐준다는 뜻. 가까워서 받쳐 주는 게 더 확실하니까요.
- $\alpha = 1/D$: 여기서 D 는 물체를 감싸는 상자의 대각선 길이¹⁷. h 를 D 로 나눠 “물체 크기 대비 얼마나 먼가”로 바꿉니다.

한 줄 요약. w_{ij} 는 “처진 면 i 가 받침면 j 로부터 받는 받침의 세기”입니다. 많이 처질수록 크고, 받침이 멀수록 작아집니다.

4.3 3단계 — 모든 받침 관계를 하나로 모으기 (흐름 텐서)

이제 물체 안의 모든 (처진 면 i , 받침면 j) 짝에 대해 위 영향력을 모읍니다.

$$F(n) = \sum_{(i,j) \in \mathcal{R}(n)} w_{ij}(n) A_i A_j m_i m_j^T \quad (6)$$

한 줄씩 읽기.

- $\sum_{(i,j) \in \mathcal{R}(n)}$: 레이캐스트로 찾은 모든 유효한 받침 짝¹⁹에 대해 더한다.

¹⁵레이캐스트(ray casting): 한 점에서 특정 방향으로 가상의 빛줄기(ray)를 쏘아, 그 빛이 처음 부딪히는 물체를 찾는 기법. 게임에서 “충을 쏘면 무언가에 맞나”, 햇빛 그림자 계산 등에 쓰입니다. 여기서는 “처진 면에서 아래로 빛을 쏘면 어느 면이 받아 주나”를 찾습니다.

¹⁶중심점(centroid): 삼각형의 한가운데 점(세 꼭짓점의 평균 위치). 면 전체를 대표하는 위치로 씁니다.

¹⁷bbox 대각선(bounding-box diagonal): 물체를 딱 감싸는 직육면체 상자의 대각선 길이. 물체의 “전체 크기”를 나타내는 대표 길이입니다. 높이차 h 를 이 크기로 나눠 줌으로써, 큰 물체든 작은 물체든 같은 기준으로 거리를 비교할 수 있게 됩니다(단위 맞추기, 정규화¹⁸).

¹⁸정규화(normalization): 서로 크기가 다른 값들을 같은 잣대로 비교할 수 있게 비율로 바꾸는 것. 예: 시험마다 만점이 다르면 100점 만점으로 환산해 비교.

¹⁹ $\mathcal{R}(n)$: 출력 방향 n 에서 “처진 면 $\rightarrow$ 받침면”으로 인정된 (i, j) 짝들의 모임. 위 세 조건(자기 자신 아님, 아래에 있음, 위를 봄)을 통과한 쌍만 들어갑니다.

- w_{ij} : 방금 구한 받침 세기.
- $A_i A_j$: 두 면이 둘 다 클수록 더 중요하게(면적 가중).
- $m_i m_j^T$: 식 (2)에서처럼 두 면의 방향 정보를 “표”로 묶은 것. 단, 여기서는 치진 면과 받침면이 서로 다른 면이라 $m_i m_j^T$ 가 됩니다(자기 자신끼리인 $m_i m_i^T$ 가 아님).

“비대칭”이 핵심. 앞의 M (식 (2))은 위아래를 구별 못 하는 대칭이었습니다. 하지만 $F(n)$ 은 “치진 면 $\rightarrow$ 받침면”이라는 방향이 들어 있어서 비대칭^a입니다. 그래서 출력 방향 n 을 바꾸면 $F(n)$ 도 제대로 바뀝니다. 이름에 “흐름(Flow)”이 붙은 이유입니다 — 받침이 한 방향으로 흐르는 구조를 담았으니까요.

^a비대칭(asymmetric): “A에서 B로”와 “B에서 A로”가 다른 성질. 물이 높은 데서 낮은 데로만 흐르듯, 받침도 위에서 아래로만 일어납니다. 이 방향성을 담았기에 위/아래를 구별할 수 있습니다.

4.4 보너스 — 계산을 너무 무겁지 않게 (레이 예산)

면이 100만 개인 큰 모델에서 모든 치진 면마다 빛을 쏘면 너무 느립니다. 그래서 쏘는 빛의 개수에 상한을 둡니다.

$$K_{ray} = \text{clip}(0.02 N_f, 500, 3000) \quad (7)$$

읽기: N_f 는 면의 개수. “전체 면의 2%(=0.02 N_f)만큼만 빛을 쏘되, 너무 적지도(최소 500개) 너무 많지도(최대 3000개) 않게 자른다”는 규칙입니다. $\text{clip}(x, a, b)$ ²⁰이 그 “범위 안으로 가두기” 함수입니다. 덕분에 모델이 아무리 커져도 한 방향당 계산량이 3000개를 넘지 않아 큰 모델도 감당할 수 있습니다.

5 가장 중요한 항: 바닥판(Ground Node)

치진 면이 아래로 빛을 쏘는데 받쳐 줄 면이 하나도 없으면 어떻게 될까요? 결국 프린터 바닥판(build plate)이 맨 밑에서 받쳐 줘야 합니다. 이 “바닥에서 올라오는 긴 기둥”이 사실 서포트의 가장 큰 부분이라, 이 항이 가장 중요합니다.

5.1 바닥판 받침의 양

$$z_{plate}(n) = \min_{v \in V} v \cdot n, \quad h_{iP}(n) = c_i \cdot n - z_{plate}(n) \quad (8)$$

한 줄씩 읽기.

- $z_{plate} = \min_{v \in V} v \cdot n$: 모든 꼭짓점²¹ 중 출력 방향으로 가장 낮은 위치 = 바닥판이 닿는 높이.
- $h_{iP} = c_i \cdot n - z_{plate}$: 치진 면 i 가 바닥에서 얼마나 높이 떠 있나 = 바닥까지 세워야 할 기둥의 길이.

$$B(n) = \sum_{i \in \mathcal{B}(n)} O_i(n) A_i (1 + \alpha h_{iP}(n)) \quad (9)$$

한 줄씩 읽기.

- $\sum_{i \in \mathcal{B}(n)}$: 받침면을 못 찾은 치진 면들만 모아 더한다²².

²⁰clip(자르기): 값 x 가 하한 a 보다 작으면 a 로, 상한 b 보다 크면 b 로 맞춰 $[a, b]$ 범위 안으로 가두는 함수. 불륨을 0~100 사이로만 조절되게 막아 두는 것과 같습니다.

²¹ V 는 메시의 모든 꼭짓점(vertex)들의 모임, v 는 그중 하나. min(최소)은 “가장 작은 값”을 고르라는 뜻.

²² $\mathcal{B}(n)$: 레이캐스트로 받침면을 찾지 못해 결국 바닥판이 받쳐야 하는 치진 면들의 모임.

- $O_i(n) A_i$: 심하게 쳐질수록, 면이 클수록 비용 ↑.
- $(1+\alpha h_{iP})$: 식 (5)의 분모와 닮았지만 여기서는 곱으로 들어갑니다. 바닥에서 높이 떠 있을수록 더 긴 기둥이 필요하니 비용이 커지는 게 맞습니다.

한 줄 요약. $B(n)$ 은 “바닥판에서 직접 받쳐 올려야 하는 서포트의 총량” 입니다. 뒤에서 보겠지만 이 항이 서포트 양을 가장 잘 예측합니다.

5.2 멋진 통합: 바닥판도 “가상의 받침면”으로 보기

논문의 핵심 아이디어 중 하나는, 위에서 따로 만든 바닥판 항 $B(n)$ 이 사실 “법선이 출력 방향 n 인 가상의 받침면을 하나 추가”한 것과 정확히 같다는 점입니다. 즉 식 (6)의 흐름 텐서에 바닥판을 더해 줍니다.

$$\tilde{F}(n) = \underbrace{\sum_{(i,j) \in \mathcal{R}(n)} w_{ij}(n) A_i A_j m_i m_j^T}_{\text{면끼리 받침(식 (6))}} + \underbrace{\sum_{i \in \mathcal{B}(n)} A_i (1 + \alpha h_{iP}) m_i n^T}_{\text{바닥판 = 가상 받침면 } n} \quad (10)$$

읽기: 두 번째 합에서 받침면의 방향 자리에 진짜 면 m_j 대신 출력 방향 n 이 들어간 것($m_i n^T$)만 다릅니다. “바닥판은 출력 방향에서 봤을 때 완벽히 위를 보는 받침면”이라고 본 것입니다.

5.3 레일리 값: 텐서를 하나의 비용 숫자로

표(텐서) $\tilde{F}$ 를 다시 하나의 비용 점수로 바꿔야 비교가 됩니다. 이때 쓰는 게 레일리 값²³입니다.

$$\tilde{R}(n) = \max(0, -n^T \tilde{F}_s(n) n) = R(n) + B(n) \quad (11)$$

한 줄씩 읽기.

- $n^T \tilde{F}_s n$: 통합 텐서를 출력 방향 n 으로 양쪽에서 눌러 짤 하나의 숫자²⁴.
- 앞의 마이너스와 $\max(0, \cdot)$: 식 (3)과 같은 정신 — “서포트가 필요한(나쁜) 양만 0 이상으로 센다”.
- $= R(n) + B(n)$: 놀랍게도, 이렇게 통합 텐서로 한 번에 계산한 값이 “면끼리 받침 비용 R ” 더하기 “바닥판 비용 B ”와 정확히 같습니다.

왜 이게 중요한가. B 를 그냥 “땀질로 끼워 넣은 보정값”이 아니라, 같은 텐서 수학에서 자연스럽게 나오는 한 부분임을 보인 것입니다. 논문은 이 등식이 컴퓨터 계산에서도 소수점 아래 10^{-12} (0.000000000001) 수준까지 정확히 맞아떨어짐을 확인했습니다. 이것이 이 방법을 “텐서장”이라 부를 자격을 줍니다.

6 최종 점수와 후보 고르기

6.1 기본 점수 J

실제로 쓰는 기본 점수는 다음과 같습니다.

²³ 레일리 몫(Rayleigh quotient): 정사각형 표(행렬) F 와 방향 n 에 대해 $n^T F n$ 으로 계산하는 하나의 숫자. 직관적으로는 “그 표가 방향 n 쪽으로 얼마나 강하게 작용하나”를 재는 값입니다. 식의 양쪽에서 n 으로 “눌러 짜낸다(contraction)”고 생각하면 됩니다. PCA의 고유값도 사실 이 레일리 값을 가장 크게/작게 만드는 방향에서 나옵니다 — 그래서 (2)의 PCA 이야기와 한 가족입니다.

²⁴ $\tilde{F}_s$ 의 아래첨자 s 는 “대칭 부분(symmetric part)”만 쓴다는 표시입니다. 레일리 값 계산에는 표의 대칭 부분만 기여하기 때문입니다. 세부는 넘어가도 좋습니다.

$$J(n) = \tilde{R}(n) + P(n), \quad P(n) = \sum_{(i,j) \in \mathcal{R}(n)} O_i(n) A_i A_j \quad (12)$$

읽기: $\tilde{R} = R + B$ 는 방금 구한 “레일리 받침 비용(면끼리 + 바닥판)”. P 는 “면끼리 받침의 양”을 따로 센 항입니다(높이 감쇠를 뺀 순수한 오버행 $\times$ 면적의 합). 둘을 더해 “이 방향에서 서포트가 대략 얼마나 들지”를 한 숫자 $J(n)$ 으로 냅니다. J 가 작을수록 좋은(서포트 적은) 방향입니다.

6.2 후보를 만드는 절차

1. 넓게 훑기: 구 표면에 고르게 뿌린 512개 방향 (피보나치 구²⁵)에서 J 를 계산.
2. 좁혀 들어가기: 점수가 좋았던 방향들 주변 $2^\circ \sim 8^\circ$ 의 좁은 원뿔 안을 더 촘촘히 다시 계산.
3. 겹치는 후보 정리: 거의 같은 방향이 여러 개 뿔히면 하나만 남기는 비최대 억제²⁶로 서로 떨어진 대표 후보들만 남김.

6.3 순위 정규화로 최종 정렬 (단위가 다른 값들을 공정하게)

마지막에는 $[J, R, P, B, H, S, \sigma_1]$ 같은 여러 특징²⁷을 함께 써서 후보 순위를 매깁니다. 그런데 이 값들은 단위가 제각각(어떤 건 부피, 어떤 건 개수)이라 그냥 더하면 불공정합니다. 그래서 각 특징을 순위(등수)로 바꿔서 비교합니다.

순위 정규화(rank normalization)란? 키 175 cm, 몸무게 70 kg처럼 단위가 다른 값을 직접 더할 순 없습니다. 대신 “키 등수 3등, 몸무게 등수 5등”처럼 등수로 바꾸면 단위가 사라져 공정하게 합칠 수 있습니다^a. SFTF도 각 특징을 후보들 사이의 등수로 바꾼 뒤 가중합해 최종 순위를 정합니다. (이는 정밀한 기하 최적화가 아니라, 경험적으로 좋은 순서를 찾는 보정입니다.)

^a이렇게 값을 등수로 바꿔 분석하는 것은 통계학의 표준 기법입니다. 뒤에 나오는 “스피어만 상관”도 같은 정신입니다.

7 후보를 “검증”하고, 어려우면 “더 본다” (AVE)

7.1 국소 정밀검증

SFTF가 추천한 상위 K 개 방향 각각의 둘레 $\pm 10^\circ$ 만 정밀 계산기 (TOMO²⁹)로 꼼꼼히 검사하고, 그중 가장 좋은 값을 고릅니다. “전부 다”가 아니라 “유망한 곳 둘레만” 보는 게 빠름의 비결입니다.

핵심 성과. 상위 20개 후보의 둘레만 검증해도, 전체 격자의 단 3.57%만 보고서 최적 대비 평균 1.13배(최악 1.58배)의 서포트 양을 찾아냈습니다. 같은 예산을 “아무 데나/규칙적으로” 쓴 비교군(평균 1.64~2.62배)보다 훨씬 좋습니다 — 어디를 볼지 잘 찍는다는 증거입니다.

²⁵피보나치 구(Fibonacci sphere): 공 표면에 점들을 최대한 고르게 흩뿌리는 잘 알려진 방법. 해바라기 씨앗이 촘촘하고 균일하게 박힌 배열과 같은 수학(황금비)을 씁니다. 덕분에 적은 점으로도 모든 방향을 골고루 시험할 수 있습니다.

²⁶비최대 억제(NMS, Non-Maximum Suppression): 비슷한 위치에 여러 후보가 몰려 있을 때 가장 좋은 것 하나만 남기고 나머지는 지우는 정리 기법. “서로 충분히 떨어진” 다양한 후보들만 남겨, 한곳에 쏠리지 않게 합니다.

²⁷여기서 H =받침을 찾은 횟수, $S \cdot \sigma_1$ =텐서에서 나오는 보조 수치(특잇값²⁸)입니다. 세부는 몰라도 됩니다 — 핵심은 “여러 종류의 단서를 함께 본다”는 점입니다.

²⁸특잇값(singular value): 행렬이 방향마다 얼마나 늘이고 줄이는지를 나타내는 수치들. 가장 큰 것을 σ_1 로 적습니다.

²⁹TOMO: 이 연구에서 진짜 서포트 양을 정밀하게 재는 기준 계산기 (검증용·비교용). CPU판과 GPU(CUDA)판이 있습니다. SFTF는 이 느린 계산기에 넘길 방향의 개수를 줄여 주는 역할입니다.

7.2 어려운 경우 자동 확대 (AVE)

가끔 정답이 아주 좁은 골짜기에 숨어 있어, 상위 10개만 봐서는 놓치는 “어려운 케이스”가 있습니다(예: *Happy* 모델). 이를 위해 **AVE**³⁰라는 규칙을 둡니다. 검증 결과가 미덥지 않다는 신호가 잡힐 때만 자동으로 후보를 더 넓혀 검사합니다.

$$\text{trigger}_{\text{AVE}} = ([\eta \geq 0.03] \wedge [\text{edge} = 1 \vee g \leq 1.5]) \vee [\text{edge} = 1 \wedge g \geq 3.0],$$

$$\eta = \frac{V_{ss}^{\text{loc}}}{V_{\text{bbox}}}, \quad g = \frac{V_{ss}^{\text{seed}}}{V_{ss}^{\text{loc}}} \quad (13)$$

한 줄씩 읽기 (기호 $\wedge$ = “그리고 (and)”, $\vee$ = “또는 (or)”³¹):

- $\eta = V_{ss}^{\text{loc}}/V_{\text{bbox}}$: 찾은 서포트 양을 물체 상자 부피로 나눈 “상대적 크기”. $\eta \geq 0.03$ 이면 “서포트가 꽤 많네?” 하는 신호.
- $g = V_{ss}^{\text{seed}}/V_{ss}^{\text{loc}}$: 처음 찍은 값 대비 검증 후 값의 비율. 검증으로 많이 좋아졌다면 (g 큼) 처음 추천이 부실했다는 뜻.
- $\text{edge} = 1$: 찾은 최적 칸이 검사 창문 가장자리에 걸쳐 있음 = “창문 밖에 더 좋은 게 있을지도?” 하는 의심 신호.
- 전체: 이런 의심 신호들이 조합되면 검사 범위를 자동 확대.

AVE는 손해 볼 일이 없습니다. 확대는 후보를 더 볼 뿐이라, 예산 (계산량)은 더 쓸 수 있어도 이미 찾은 답이 나빠질 수는 없습니다. 실제로 어려운 케이스의 최악 비율을 **8.52배** → **1.06배**로 끌어내렸고 (평균 예산은 3~4.5%), 다섯 그룹 점검에서도 최악 **17.57배** → **1.37배**로 좋아졌습니다.

8 결과 요약과 정직한 한계

8.1 얼마나 빠르나

같은 1° 격자 조건에서, SFTF는 정밀 계산기 (TOMO_CPU) 대비 **파이썬 구현으로 8~382배, C++ 구현으로 최대 5만 6천 배까지** 빨랐습니다. 빠름의 핵심은 “계산기를 바꿔서”가 아니라 “검사할 방향의 개수 자체를 확 줄여서”입니다. 식 (7)의 레이 예산 덕분에 면이 100만 개인 거대 모델도 감당했습니다.

8.2 어떤 향이 가장 끌모 있나

여러 특징과 “진짜 서포트 양”의 스피어만 상관³²을 재 보니, **바닥판 향 B 가 가장 강한 단일 예측자**였습니다 (평균 0.58). 통합 레일리 향 $\tilde{R} = R + B$ 는 더 높은 0.60이었고요. 이는 “바닥판 받침이 서포트 양을 지배한다”는 직관과 정확히 맞습니다.

³⁰ AVE (Adaptive Verification Escalation, 적응적 검증 확대): “검사해 보니 확신이 안 서면, 그때만 검사 범위를 자동으로 넓히는” 규칙. 평소엔 적게 보고, 미심쩍을 때만 더 보는 똑똑한 절약 장치입니다.

³¹ 논리 기호: $\wedge$ 는 두 조건이 모두 참일 때 참 (“그리고”), $\vee$ 는 하나라도 참이면 참 (“또는”). $[\cdot]$ 는 “안의 조건이 맞으면 1(참), 아니면 0(거짓)”.

³² 스피어만 상관계수 (Spearman rank correlation): 두 양이 “같이 커지고 같이 작아지는지”를 등수 기준으로 재는 값 ($-1 \sim 1$). +1에 가까우면 “한쪽이 크면 다른 쪽도 크다” (강한 양의 관계), 0이면 무관, -1이면 정반대. 앞서 본 순위 정규화처럼 값을 등수로 바꿔 보기 때문에, 단위나 곡선 모양에 휘둘리지 않고 “경향”만 봅니다.

8.3 정직한 한계 (이게 제일 중요)

SFTF는 “정밀 최적화기”가 아닙니다. 순위만으로 정답을 내는 점수는, 한 번도 본 적 없는 새 모델에서는 가끔 크게 빗나갑니다(교차검증⁴에서 최악 비율이 100배를 넘는 경우도 있었습니다).

⁴교차검증 (cross-validation): 가진 데이터의 일부를 일부러 가려 두고 나머지로만 규칙을 만든 뒤, 가려 둔 것으로 시험해 “처음 보는 데이터에도 통하는가”를 점검하는 방법. 시험 범위만 외운 게 아니라 진짜 실력인지 확인하는 모의고사와 같습니다. 특히 한 모델씩 빼고 시험하는 방식을 LOMO³³라고 합니다.

³³LOMO (Leave-One-Mesh-Out): 메쉬(모델)를 하나씩 차례로 빼 두고 나머지로 학습 · 그 하나로 시험하기를 반복하는 교차검증 방식.

그래서 이 방법의 올바른 사용법은 처음 말한 그대로입니다 — **값싼 후보 추천 (SFTF)** → **유망한 곳만 정밀 검증 (TOMO)** → **어려우면 자동 확대 (AVE)**. 이 조합으로만 신뢰할 수 있는 결과가 나옵니다. SFTF는 정밀 설계를 대체하는 게 아니라, 비싼 검증을 어디에 집중할지 빠르게 알려 주는 똑똑한 **출발점 (warm start)**입니다.

전체 한 줄 정리. “서포트 적은 출력 방향 찾기”라는 비싼 문제에서, SFTF는 받침을 흐름으로 보는 텐서로 유망한 방향을 **빠르고 값싸게** 추려 주고 (특히 바닥판 항 B 가 핵심), 그 둘레만 정밀 검증하며, 어려운 경우엔 자동으로 더 들여다봅니다. 똑똑하게 “어디를 볼지” 잘 짚어 주는 도구입니다.

원문: *SFTF_draft.tex* (Support Flow Tensor Field for Fast Build-Orientation Candidate Generation).

이 글은 같은 내용을 고등학생 눈높이로 풀어 쓴 해설본입니다.